

How and When To Write a Template

Roth Michaels

2024

Roth Michaels
Principal Software Engineer
Native Instruments



NATIVE INSTRUMENTS^{NI}



iZOTOPE



Plugin Alliance



BRAINWORX

01:00:39:02.62 44,1 Varispeed 120.0000 Tempo beibehalten 4/4 /16 Kein In Kein Out CPU 0% Disk 0% i234

Bearbeiten Funktionen Ansicht 9 17 25 33 41 49 57 65 73 81 89 97 105 113 121 129 137 145

1 Norweg...d_audio
M S R I

2 Inst 1
M S R

3 Vogel
M S R I

4 Wind_Audio
M S R I

5 Dub chords
M S R I

6 Inst 2
M S R

7 Inst 3
M S R

8 Inst 4
M S R

9 Dub ch...alltime
M S R I

10 Molek Melodies
M S R I

11 goss01 Merged.2
M S R I

12 Bongos plus 100
M S R I

13 Low Ris...is plings
M S R I

14 drums
M S R I

19 Big 808
M S R I

20 808 Bass
M S R I

21 mm wave weaver
M S R I

22 norweg...S JAZZ
M S R I

23 norweg...o spirit
M S R I

24 pads
M S R I

regian Wood_audio.1 (C)

Norwegian Wood_audio.2 (C)

Inst 3

Manuell

Vergleichen Wiederrufen Wiederholen Ansicht: Editor

7 SKIES, Magnificence - THE DRILL

Output: st.1 Voices: 0 Hps: 32 Purge

HEX Ch: [N] 1 Memory: 43.09 MB

CPU 0% Disk 0%

gr808

808 Center

808 Sides

Mode [mono]

[attack: fast] [release: fast]

Panning

Reverb: [Cloud Grain]

Size

Amount

Rev Lo-Cut: 20 20K Rev Hi-Cut: 20 20K

Bouncy FX: [off]

Prenets: 1 2 3

Speed ModW:

gr808 kick

Kontakt

Inst 2

Manuell

Vergleichen Wiederrufen Wiederholen Ansicht: Editor

MS20 Drums

Output: st.1 Voices: 0 Hps: 32 Purge

HEX Ch: [N] 1 Memory: 14.02 MB

CPU 0% Disk 0%

Cutoff Saturation Reverb Delay Delay Time

Peter Flint: MS20 Drums

Kontakt

Bongos plus 100.2 (C)

Low Risk Pharmaceuticals plings.13 (C)

norwegian woods mm RHODES JAZZ (C)

norwegian woods mm RHODES JAZZ (C)

norwegian woods mm atmo spirit.1 (C)

norwegian woods mm pads.1 (C)

norwegian woods mm pads.3 (C)

norwegian woods mm pads.5 (C)

norwegian woods mm pads.6 (C)

You can do it!



#include <cpp>

<https://www.incldecpp.org>

Why I wanted to give this talk

Why I wanted to give this talk

My favorite interview question...

Let's make this talk interactive!

...please ask questions

Generic programming with templates

```
auto msg = std::string{"hello, world"};  
auto it = std::find(begin(msg), end(msg), ',');
```

Generic programming with templates

```
auto msg = std::string{"hello, world"};  
auto it = std::find(begin(msg), end(msg), ' ');  
  
auto v = std::vector{1, 3, 42, 0, 50};  
auto it = std::find(begin(v), end(v), 42);
```

Generic programming with templates

```
auto msg = std::string{"hello, world"};  
auto it = std::find(begin(msg), end(msg), ',');  
  
auto v = std::vector<int>{1, 3, 42, 0, 50};  
auto it = std::find(begin(v), end(v), 42);
```

Generic programming with templates

```
auto msg = std::basic_string<char>{"hello, world"};  
auto it = std::find(begin(msg), end(msg), ' ', '');
```

```
auto v = std::vector<int>{1, 3, 42, 0, 50};  
auto it = std::find(begin(v), end(v), 42);
```

What are templates?

...recipes for generating specializations.

A recipe

Ingredients for berry bars

- 4 cups of fresh or frozen berries
- 1/2 cup granulated sugar
- 2 tablespoons cornstarch
- 1 tablespoon lemon juice

A recipe

Ingredients for <berry> bars

- 4 cups of fresh or frozen berries
- 1/2 cup granulated sugar
- 2 tablespoons cornstarch
- 1 tablespoon lemon juice

A recipe

Ingredients for <blueberry> bars

- 4 cups of fresh or frozen blueberries
- 1/2 cup granulated sugar
- 2 tablespoons cornstarch
- 1 tablespoon lemon juice

A recipe

Ingredients for <raspberry> bars

- 4 cups of fresh or frozen raspberries
- 1/2 cup granulated sugar
- 2 tablespoons cornstarch
- 1 tablespoon lemon juice

Types of templates

- class templates
- function templates
- alias templates (C++11)
- variable templates (C++17)
- lambda templates (C++20)

Types of templates

- class templates
- function templates
- alias templates (C++11)
- variable templates (C++17)
- lambda templates (C++20)
- (concepts) (C++20)
- family of classes
- family of functions
- family of aliases
- family of variables
- family of lambdas

Templates Made Easy With C++20

Roth Michaels



Why do we want to use templates?

...don't repeat yourself—a class template example

```
class OptionalInt {
public:

    OptionalInt() = default;

    explicit OptionalInt(int value)
        : m_value{value}, m_active{true} {}

    int operator*() { return m_value; }

private:
    int m_value{};
    bool m_active{false};
};
```

```
class OptionalFloat {  
public:  
  
    OptionalFloat() = default;  
  
    explicit OptionalFloat(float value)  
        : m_value{value}, m_active{true} {}  
  
    int operator*() { return m_value; }  
  
private:  
    float m_value{};  
    bool m_active{false};  
};
```

```
template <typename T>
class Optional {
public:

    Optional() = default;

    explicit Optional(T value)
        : m_value{value}, m_active{true} {}

    T operator*() { return m_value; }

private:
    T m_value{};
    bool m_active{false};
};
```

```
template <typename T>
class Optional {
public:

    Optional() = default;

    explicit Optional(T value)
        : m_value{value}, m_active{true} {}

    T operator*() { return m_value; }

private:
    T m_value{};
    bool m_active{false};
};
```

```
template <class T>
class Optional {
public:

    Optional() = default;

    explicit Optional(T value)
        : m_value{value}, m_active{true} {}

    T operator*() { return m_value; }

private:
    T m_value;
    bool m_active{false};
};
```

```
template <class T>
class Optional {
public:

    Optional() = default;

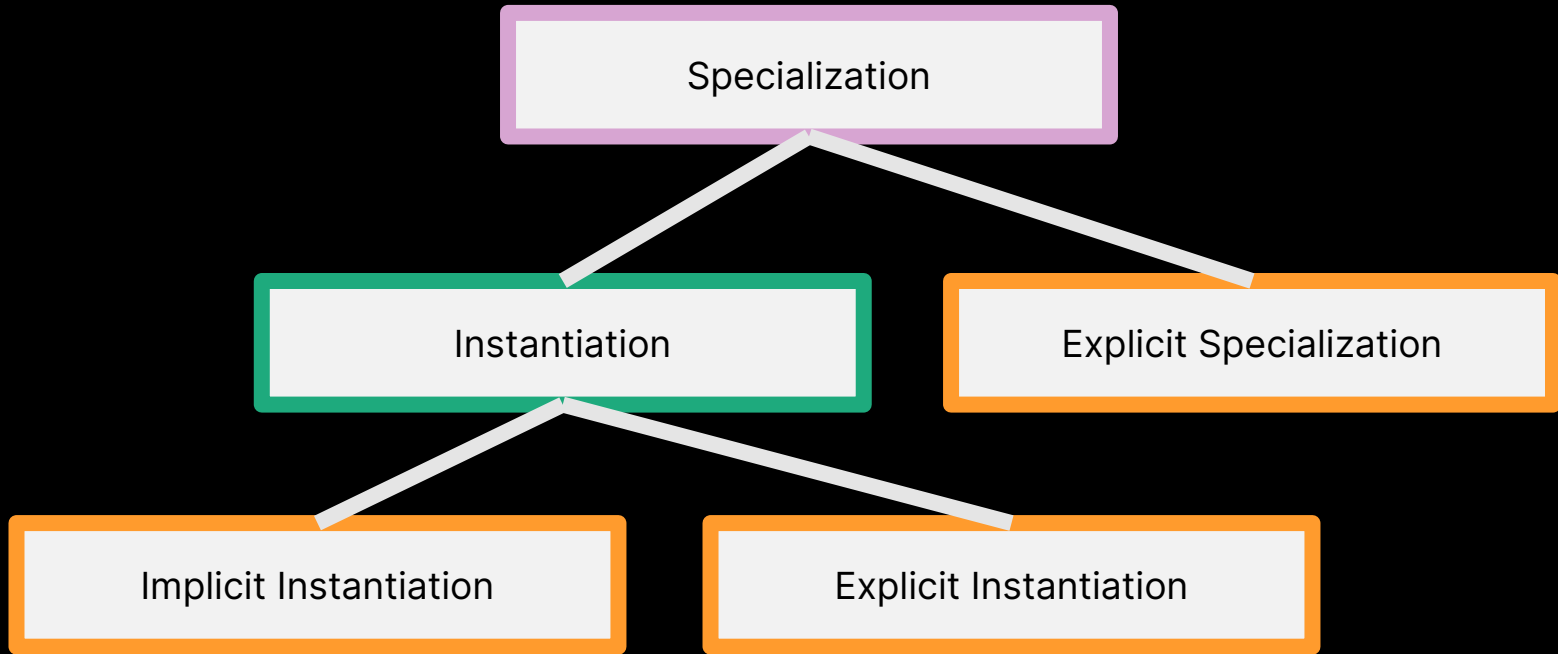
    explicit Optional(T value)
        : m_value{value}, m_active{true} {}

    T operator*() { return m_value; }

private:
    T m_value;
    bool m_active{false};
};
```

How are templates instantiated?

Let's create a class template specialization.



First the template is compiled

```
template <class T>
class Optional {
public:
    Optional() = default;
    explicit Optional(T value)
        : m_value{value}, m_active{true} {}
    T operator*() { return m_value; }

private:
    T m_value;
    bool m_active{false};
};
```

First the template is compiled

```
auto foo = Optional<int>{};
```

First the template is compiled

```
auto foo = Optional<int>{};
```

```
template <>
class Optional<int> {
public:
    Optional() = default;
    explicit Optional(int value)
        : m_value{value}, m_active{true} {}
    int operator*() { return m_value; }

private:
    int m_value;
    bool m_active{false};
};
```

Function, alias, and variable templates

Function templates

```
template <class T>
void my_print(const T& x) {
    std::cout << x << '\n';
};
```

Function templates

```
void my_print(const auto& x) {  
    std::cout << x << '\n';  
};
```

Alias templates

```
template <class T>  
using deck = std::deque<T>;
```

```
template <class T>  
using dq = std::deque<T>;
```

Variable templates

```
namespace std {  
  
    template <class T>  
    bool std::is_floating_point_v =  
        is_floating_point<T>::value;  
  
}
```

Variable templates

```
namespace std {  
  
template <class T>  
bool std::is_floating_point_v =  
    is_floating_point<T>::value;  
  
}
```

An aside on metaprogramming

...too much for this talk.

```
template <unsigned N> struct Fibonacci {  
    static constexpr auto value =  
        Fibonacci<N - 2>::value + Fibonacci<N - 1>::value;  
};  
  
template <> struct Fibonacci<1> {  
    static constexpr unsigned value = 1u;  
};  
  
template <> struct Fibonacci<0> {  
    static constexpr unsigned value = 0u;  
};  
  
static_assert(Fibonacci<6u>::value == 8u);
```

Abstract

- “Template metaprogramming has become an important part of a C++ programmer's toolkit. This talk will demonstrate state-of-the-art metaprogramming tools and techniques, applying each to obtain representative implementations of selected standard library facilities.
- “Along the way, we will look at `void_t`, a recently-proposed, extremely simple new `<type_traits>` candidate whose use has been described by one expert as ‘highly advanced (and elegant), and surprising even to experienced template metaprogrammers.’ ”
- Presented in two parts, with a short break between.
- **Note:** not intended for C++ novices! (Sorry; another time?)

Copyright © 2014 by Walter E. Brown. All rights reserved.

2



<https://www.youtube.com/watch?v=Am2is2QCvxY>

Modern Template Metaprogramming (Walter Brown)



Back to our story...

...let's see a variable template.

Variable templates

```
template <class T>  
constexpr T PI =  
    T(3.1415926535897932385L);
```

```
template <class T>  
auto circumference(T rayon) {  
    static_assert(std::is_floating_point_v<T>);  
    return T(2) * PI<T> * rayon;  
}
```

Variable templates

```
template <std::floating_point T>  
constexpr T PI =  
    T(3.1415926535897932385L);
```

```
template <std::floating_point T>  
auto circumference(T rayon) {  
  
    return T(2) * PI<T> * rayon;  
}
```

Default template parameters

Non-type template parameters

```
template  
<class T,  
    class Allocator = std::allocator<T>>  
class vector;
```

Variadic templates

Variadic templates

```
template <class... Types>  
class variant;
```

Variadic templates with fold expressions (C++20)

```
template <class... Args>  
constexpr auto sum(Args... args) {  
    return (... + args);  
}  
  
static_assert(sum(1, 2, 3, 4) == 10);
```

Variadic templates (pre-fold expressions)

```
template <class T>
```

```
T sum(T x) {  
    return x;  
}
```

```
template <class First, class... Rest>  
auto sum(First first, Rest... rest) {  
    return first + sum(rest...);  
}
```

```
static_assert(Sum(1, 2, 3, 4) == 10);
```

How does type deduction
work?

Template argument deduction

```
auto msg = std::string{"hello, world"};  
auto it = std::find(begin(msg), end(msg), ' ', ' ');  
  
auto v = std::vector<int>{1, 3, 42, 0, 50};  
auto it = std::find(begin(v), end(v), 42);
```

Template argument deduction

```
auto msg = std::string{"hello, world"};  
auto it = std::find(begin(msg), end(msg), ' ', ' ');  
  
auto v = std::vector{1, 3, 42, 0, 50};  
auto it = std::find(begin(v), end(v), 42);
```

How does type deduction work?

...demo time! <https://cppinsights.io/s/966a55b4>

Templates can be parameterized on more than types...

...NTTP and template template parameters.

Non-type template parameters

```
template <class T, std::size_t N>  
struct array;
```

Non-type template parameters

```
template <class T, size_t N>
std::array<T, N> stripAtomic(
    const std::array<std::atomic<T>, N>& input) {

    std::array<T, N> out;
    std::copy(begin(input), end(input), begin(out));
    return out;
};

auto atomicData = std::array<std::atomic<float>, 4>{
    1.2f, 0.f, -2.f, 0.f};

auto data = stripAtomic(atomicData);
```

Template template parameters

```
template <class T, template <class> typename Allocator>
struct TwoContainers {
    std::list<T, Allocator<T>> list;
    std::vector<T, Allocator<T>> vec;
};

auto c = TwoContainers<float, std::allocator>{};
```

Template template parameters

```
template <class T, template <class> typename Allocator>
struct TwoContainers {
    std::list<T, Allocator<T>> list;
    std::vector<T, Allocator<T>> vec;
};

auto c = TwoContainers<float, std::allocator>{};
```

Why must I type **typename**?

...NTTP and template template parameters.

T::One

```
template <class T, size_t N>
struct Thing {
    using One = T;
    static const size_t Two{N};
};
```

```
template <class T>
struct PlaysWithCat {
    using ValueType = T::One;
    std::array<ValueType, T::Two> children;
};
```

typename.cpp:18:20: error: missing 'typename'
prior to dependent type name 'T::One'

using ValueType = T::One;

^~~~~~

typename

```
template <class T, size_t N>
struct Thing {
    using One = T;
    static const size_t Two{N};
};
```

```
template <class T>
struct PlaysWithCat {
    using ValueType = typename T::One;
    std::array<ValueType, T::Two> children;
};
```

```
template <class T, size_t N>
struct Thing {
    using One = T;
    static const size_t Two{N};
};
```

```
template <>
struct Thing<int, 2> {
    static const int One{1};
    static const int Two{2};
};
```

```
template <>
struct Thing<int, 2> {
    int One{1};
    int Two{2};
};
```

```
template <class T>
struct PlaysWithCat {
    using ValueType = typename T::One;
    std::array<ValueType, T::Two> children;
};
auto p = PlaysWithCat<Thing<int, 2>>;
```

typename.cpp:18:32: error: typename specifier refers to non-type member 'One' in

'Thing<int, 2>'

using ValueType = typename T::One;
~~~~~^~~

typename.cpp:32:11: note: in instantiation of template class 'PlaysWithCat<Thing<int, 2>>' requested here

auto p = PlaysWithCat<Thing<int, 2>>{};  
^

typename.cpp:11:23: note: referenced member 'One' is declared here

static const int One{1};

# Should you specialize functions?

...try to use overloads.

```
struct Roman {};
```

```
template <class T>  
void Foo(T lhs, T rhs) {  
    cout << "?\n";  
}
```

```
template <>  
void Foo<Roman>(Roman lhs, Roman rhs) {  
    cout << "Ecce, Romani!\n"  
}
```

```
struct Roman {};
```

```
template <class T>  
void Foo(T lhs, T rhs) {  
    cout << "?\n";  
}
```

```
void Foo(Roman lhs, Roman rhs) {  
    cout << "Ecce, Romani!\n"  
}
```

# What is SFINAE?

...use concepts instead.

What is SFINAE

```
struct Foo { using Bar = int; };
```

```
template <typename T>  
void Baz(typename T::Bar);  
template <typename T>  
void Baz(T);
```

```
Baz<int>(42); // calls second overload  
Baz<Foo>(42); // calls first overload
```

What is SFINAE

```
struct Foo { using Bar = int; };
```

```
template <typename int>  
void Baz(typename int::Bar);  
template <typename int>  
void Baz(int);
```

```
Baz<int>(42); // calls second overload
```

What is SFINAE

```
struct Foo { using Bar = int; };
```

```
template <typename int>  
void Baz(int);
```

```
Baz<int>(42); // calls second overload
```

What is SFINAE

```
struct Foo { using Bar = int; };
```

```
template <typename Foo>  
void Baz(typename Foo::Bar);  
template <typename int>  
void Baz(int);
```

```
Baz<Foo>(42); // calls first overload
```

What is SFINAE

```
template <class T, class = void>
struct HasNameFunction : std::false_type {};
```

```
template <class T>
struct HasNameFunction<T, std::void_t<decltype(T::name())>> :
std::true_type {};
```

```
template <class T>
inline constexpr bool HasNameFunction_v =
HasNameFunction<T>::value;
```

```
template <class T> auto GetName() {
    if constexpr (HasNameFunction_v<T>) {
        return T::name();
    } else {
        return T::name;
    }
};
```

# When to write a template?

...just some examples.

templates and alternatives

- Templates
- Variants
- Virtual Interfaces

# Working with lambdas

```
class BlockProcessor {
public:

    template <class Fn>
    void Process(std::span<float> buf, Fn&& fn) {
        // pre-process buf
        fn(buf);
    }
};

auto bp = BlockProcessor{};
auto data = std::vector<float>{ . . . };

bp.Process(data, [](std::span<float> buf) {
    std::transform(begin(buf), end(buf),
                   begin(buf), std::negate<float>());
});
```

```
template <class Fn>
class WrappedProcessor {
public:
    WrappedProcessor(Fn&& fn) : m_fn{std::forward<Fn>(fn)} {}

    void Process(std::span<float> buf) {
        // pre-process buf
        m_fn(buf);
    }
private:
    Fn m_fn;
};
```

```
class WrappedProcessor {
public:
    WrappedProcessor(std::function<void(std::span<float>)> fn)
        : m_fn{std::move(fn)} {}

    void Process(std::span<float> buf) {
        // pre-process buf
        m_fn(buf);
    }
private:
    std::function<void(std::span<float>)> m_fn;
};
```

# Visiting

```
using MyThings = std::variant<int, std::string,  
                                std::optional<int>>;  
  
void print_my_things(const MyThings& thing) {  
    std::visit([](auto x) {  
        if constexpr (std::same_as<decltype(x),  
                                std::optional<int>>) {  
            if (x) { std::cout << *x << '\n'; }  
            else { std::cout << "empty\n"; }  
        } else {  
            std::cout << x << '\n';  
        }  
    }, thing);  
}
```

```
using MyThings = std::variant<int, std::string,  
                                std::optional<int>>;  
  
void print_my_things(const MyThings& thing) {  
    std::visit([](auto x) {  
        if constexpr (std::same_as<decltype(x),  
                                std::optional<int>>) {  
            if (x) { std::cout << *x << '\n'; }  
            else { std::cout << "empty\n"; }  
        } else {  
            std::cout << x << '\n';  
        }  
    }, thing);  
}
```

```
using MyThings = std::variant<int, std::string,  
                                std::optional<int>>;  
  
void print_my_things(const MyThings& thing) {  
    std::visit([](auto x) {  
        if constexpr (std::same_as<decltype(x),  
                                std::optional<int>>) {  
            if (x) { std::cout << *x << '\n'; }  
            else { std::cout << "empty\n"; }  
        } else {  
            std::cout << x << '\n';  
        }  
    }, thing);  
}
```

```
using MyThings = std::variant<int, std::string,  
                                std::optional<int>>;  
  
void print_my_things(const MyThings& thing) {  
    std::visit([](auto x) {  
        if constexpr (std::same_as<decltype(x),  
                                std::optional<int>>) {  
            if (x) { std::cout << *x << '\n'; }  
            else { std::cout << "empty\n"; }  
        } else {  
            std::cout << x << '\n';  
        }  
    }, thing);  
}
```

```
using MyThings = std::variant<int, std::string,  
                                std::optional<int>>;  
  
void print_my_things(const MyThings& thing) {  
    std::visit([](auto x) {  
        if constexpr (std::same_as<decltype(x),  
                                std::optional<int>>) {  
            if (x) { std::cout << *x << '\n'; }  
            else { std::cout << "empty\n"; }  
        } else {  
            std::cout << x << '\n';  
        }  
    }, thing);  
}
```

```
using MyThings = std::variant<int, std::string,  
                                std::optional<int>>;  
  
void print_my_things(const MyThings& thing) {  
    std::visit([](auto x) {  
        if constexpr (std::same_as<decltype(x),  
                                std::optional<int>>) {  
            if (x) { std::cout << *x << '\n'; }  
            else { std::cout << "empty\n"; }  
        } else {  
            std::cout << x << '\n';  
        }  
    }, thing);  
}
```

# Visiting

...with a lambda template

```
using MyThings = std::variant<int, std::string,  
                                std::optional<int>>;  
  
void print_my_things(const MyThings& thing) {  
    std::visit([]<class T>(const T& x) {  
        if constexpr (std::same_as<T,  
                                std::optional<int>>) {  
            if (x) { std::cout << *x << '\n'; }  
            else { std::cout << "empty\n"; }  
        } else {  
            std::cout << x << '\n';  
        }  
    }, thing);  
}
```

# Templates and Testing

```
namespace TP {  
    class Activation {  
    public:  
        Activation(std::string_view);  
  
        bool IsValid() const;  
    };  
}
```

```
bool TestAuth(const std::vector<std::string>& pids) {  
    return std::any_of(begin(pids), end(pids),  
        [](const auto& pid) {  
            return TP::Activation{pid}.IsValid();  
        });  
}
```

```
namespace TP {  
    class Activation {  
    public:  
        Activation(std::string_view);  
  
        bool IsValid() const;  
    };  
}
```

```
template <class ActivationType = TP::Activation>  
bool TestAuth(const std::vector<std::string>& pids) {  
    return std::any_of(begin(pids), end(pids),  
                        [](const auto& pid) {  
                            return ActivationType{pid}.IsValid();  
                        });  
}
```

```
namespace {
    class StubActivation {
    public:
        Activation(std::string_view pid) : m_pid{pid} {}

        bool IsValid() const {
            return m_pid == "Z05";
        }
    private:
        std::string m_pid;
    };
}

TEST_CASE("Expect to find activation") {
    auto pids = std::vector<std::string>{"Z01", "Z05", "Z42"};
    REQUIRE(TestAuth<StubActivation>(pids));
}
```

```
// TestAuth.h
```

```
template <class ActivationType = TP::Activation>
bool TestAuth(const std::vector<std::string>& pids) {
    return std::any_of(begin(pids), end(pids),
        [](const auto& pid) {
            return ActivationType{pid}.IsValid();
        });
}
```

```
extern template
bool TestAuth<TP::Activation>(const std::vector<std::string>&);
```

```
// TestAuth.cpp
```

```
template
bool TestAuth<TP::Activation>(const std::vector<std::string>&);
```

# Generic Test Harness

```
template <class T>
concept InPlaceProcessor = requires (T p) {
    p.Process(std::span<float>{});
};
```

```
template <class T>
concept InPlaceProcessor = requires (T p) {
    p.Process(std::span<float>{});
};
```

```
class ExampleProcessor {
public:
    void Process(std::span<float>);
};
```

```
template <InPlaceProcessor T>
bool testProcessor(T& p) {
    std::mt19937 rng(42);
    std::uniform_real_distribution<float> dist(-1.0f, 1.0);
    std::array<float, 1024> buffer;
    std::generate(begin(buffer), end(buffer), [&] {
        return dist(rng);
    });

    p.Process(buffer);

    return std::any_of(begin(buffer), end(buffer),
        [](float x) {
            return x != 0.f;
        });
}
```

When to use templates to  
parameterize numeric types?

```
template <class T>
class OnePoleFilter {
public:
    void Process(std::span<T> buf) {
        std::transform(begin(buf), end(buf), begin(buf),
            [this](float x) {
                m_prev = T(0.01) * x + T(0.99) * m_prev;
                return m_prev;
            });
    }

    void Reset() {
        m_prev = 0.f;
    }

private:
    T m_prev(0);
};
```

```
template <std::floating_point T>
class OnePoleFilter {
public:
    void Process(std::span<T> buf) {
        std::transform(begin(buf), end(buf), begin(buf),
            [this](float x) {
                m_prev = T(0.01) * x + T(0.99) * m_prev;
                return m_prev;
            });
    }

    void Reset() {
        m_prev = 0.f;
    }

private:
    T m_prev(0);
};
```

```
template <class T>
class MySuperCoolEffect {
public:

    void Process(std::span<T> buf) {
        // process audio
        m_filter.Process(buf);
        // more processing
    }

    void Reset() {
        m_filter.Reset();
    }

private:
    OnePoleFilter<T> m_filter;
};
```

```
class MySuperCoolEffect {
public:

    void Process(std::span<float> buf) {
        // process audio
        m_filter.Process(buf);
        // more processing
    }

    void Reset() {
        m_filter.Reset();
    }

private:
    OnePoleFilter<float> m_filter;
};
```

```
class MySuperCoolEffect {
public:
    using FloatType = float;
    void Process(std::span<FloatType> buf) {
        // process audio
        m_filter.Process(buf);
        // more processing
    }

    void Reset() {
        m_filter.Reset();
    }

private:
    OnePoleFilter<FloatType> m_filter;
};
```

# Bonus example: splitting templates into multiple files

<https://cppinsights.io/s/d2407e71>

Bonus example: splitting  
templates into multiple files  
plus extern template

<https://cppinsights.io/s/8fb5e8dd>



# Thank you!

Roth Michaels

Principal Software Engineer

[roth.michaels@native-instruments.com](mailto:roth.michaels@native-instruments.com)

[@thevibesman](#)